

02_TRD

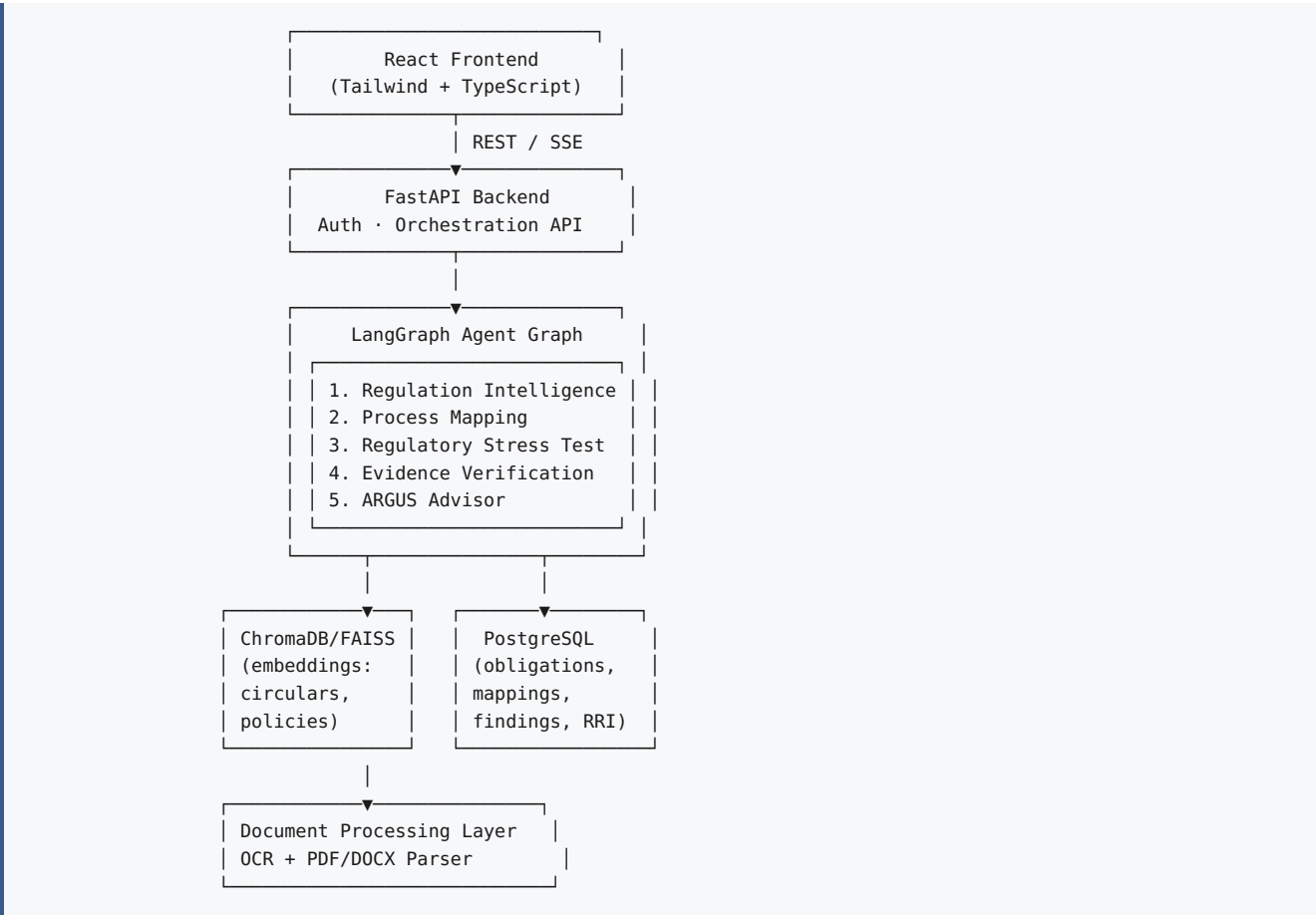
Agentic Regulatory Governance & Unified Simulation

Team Rocket

July 2026

1. Architecture Overview

ARGUS is a multi-agent system orchestrated by LangGraph, sitting on top of a RAG layer over both regulatory text and organizational documents, backed by a relational store for structured state (obligations, mappings, findings, scores) and a vector store for semantic retrieval.



2. Technology Stack

Layer	Choice	Notes
Frontend	React + TypeScript + Tailwind CSS	SPA, component-driven dashboards
Backend API	FastAPI (Python 3.11+)	Async, OpenAPI-documented
Orchestration	LangGraph	Stateful multi-agent graph, checkpointing per run
RAG	LangChain + ChromaDB/FAISS	Chunked embeddings of circulars and internal docs
LLM	Gemini / Llama / GPT (pluggable)	Abstracted behind a provider interface
Relational DB	PostgreSQL 15+	Structured entities, audit trail

Document Processing	PyPDF/pdfplumber + Tesseract OCR	Native text extraction with OCR fallback
Auth	JWT (OAuth2 password/bearer)	RBAC middleware
Deployment (hackathon)	Docker Compose	Frontend, backend, Postgres, vector DB as services

3. Agent Design (LangGraph)

Each circular processing run is a LangGraph state machine. State object carries: `circular_id`, `raw_text`, `obligations[]`, `mappings[]`, `findings[]`, `evidence_status[]`, `rri`, `errors[]`.

3.1 Node: Regulation Intelligence Agent

- **Input:** circular raw text (chunked).
- **Task:** structured extraction → `Obligation{id, description, deadline, applicability, source_ref}`.
- **Technique:** LLM function-calling with a fixed JSON schema; chunk + merge for long circulars; dedupe near-identical obligations via embedding similarity.
- **Output:** `obligations[]` written to Postgres, embedded into vector store.

3.2 Node: Process Mapping Agent

- **Input:** `obligations[]` + org’s policy/SOP corpus (pre-embedded).
- **Task:** for each obligation, retrieve top-k semantically similar policy chunks; LLM judges relevance and assigns mapping confidence + responsible department.
- **Output:** `mappings[]` (`obligation_id` → `policy_id[]`, `department`, `confidence`); obligations below confidence threshold flagged unmapped.

3.3 Node: Regulatory Stress Test Agent (core differentiator)

- **Input:** `obligations[]`, `mappings[]`, org metadata (policy last-updated dates, workflow definitions).
- **Task:** rule-assisted + LLM-reasoned checks per obligation:
 - No mapping → **Unimplemented obligation**.
 - Mapping exists but policy `last_updated` predates circular effective date → **Outdated procedure**.
 - Mapping exists but conflicting obligation mapped to same policy → **Policy conflict**.
 - Workflow step referenced but no corresponding approval/process defined → **Workflow gap**.
- **Output:** `findings[]` (`type`, `obligation_id`, `severity`, `description`).

3.4 Node: Evidence Verification Agent

- **Input:** `findings[]`, uploaded evidence corpus (classified by type: policy doc / training record / compliance report / approval / audit doc).
- **Task:** for each finding requiring evidence, retrieve and classify whether valid evidence exists and is current.
- **Output:** `evidence_status[]` (`Present` / `Stale` / `Missing`), merged into findings.

3.5 Node: RRI Calculator (deterministic, non-LLM)

- **Input:** `mappings[]`, `findings[]`, `evidence_status[]`.
- **Task:** compute weighted score:
 - Policy Alignment (25%) = mapped obligations / total obligations
 - Control Coverage (25%) = obligations w/o “Unimplemented” finding / total
 - Evidence Completeness (20%) = “Present” evidence / evidence required
 - Workflow Readiness (15%) = obligations w/o “Workflow gap” / total
 - Employee Readiness (15%) = training-record evidence present / departments affected
- **Output:** `rri` object with overall + component scores, persisted with timestamp for trend tracking.

3.6 Node: ARGUS Advisor

- **Input:** any finding + full traceability chain.
- **Task:** RAG-grounded natural-language explanation and recommended fix; also serves ad-hoc chat queries scoped to the org’s data.
- **Output:** explanation text + Regulatory Replay chain (ordered list of entities from circular → obligation → policy → workflow → gap → fix).

4. Data Flow Summary

1. Circular uploaded → parsed → chunked → embedded → stored.
2. Regulation Intelligence Agent runs → obligations persisted, human-reviewable via UI.
3. On confirmation, Process Mapping Agent runs → mappings persisted.
4. Stress Test Agent runs → findings persisted.
5. Evidence Verification Agent runs (can run incrementally as evidence is uploaded) → evidence_status updated, findings re-evaluated.
6. RRI Calculator runs on any state change → new RRI snapshot stored.
7. Advisor is invoked on-demand (per finding or via chat) — not part of the critical path.

5. API Surface (representative)

Method	Endpoint	Purpose
POST	/circulars	Upload + kick off extraction
GET	/circulars/{id}/obligations	List extracted obligations
PATCH	/obligations/{id}	Human edit/confirm obligation
POST	/circulars/{id}/map	Trigger Process Mapping Agent
POST	/circulars/{id}/stress-test	Trigger Stress Test Agent
POST	/evidence	Upload evidence artifact
GET	/readiness/{org_id}	Current RRI + trend
GET	/findings/{id}/replay	Regulatory Replay chain for a finding
POST	/advisor/query	Ask ARGUS Advisor a question
GET	/reports/{circular_id}	Generate/download audit-ready PDF

6. Non-Functional / Engineering Concerns

- **Idempotency:** re-running an agent node for the same circular must be safe (upsert, not duplicate, obligations/mappings).
- **Human-in-the-loop gate:** obligations must be confirmable/editable before they feed downstream agents — prevents compounding extraction errors.
- **Provider abstraction:** LLM calls go through a single interface so Gemini/Llama/GPT can be swapped without touching agent logic.
- **Traceability:** every finding and RRI component must store references back to source obligation IDs and policy/evidence document IDs (required for Regulatory Replay and audit export).
- **Tenant isolation:** all queries scoped by org_id; Postgres row-level security recommended for multi-tenant deployment beyond hackathon.
- **Cost/latency control:** cache embeddings; avoid re-embedding unchanged documents; batch LLM calls per chunk rather than per obligation where possible.

7. Deployment (Hackathon MVP)

- Docker Compose services: frontend, backend, postgres, chromadb.
- Environment-based config for LLM provider API keys.
- Seed script to load sample SEBI circular + synthetic org policy corpus for demo.